Hybrid-INoT: a hybrid agent architecture with internal role segregation for engineering tasks under long shared context

Daniil Privezentsev[1*]

[1*]National Research University Higher School of Economics, Moscow, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Abstract

External multi-agent orchestration built on large language models repeatedly passes the shared context to independent roles and thereby creates substantial coordination overhead. In software engineering tasks with long shared context, this leads to rapid growth in token usage. This paper studies **Hybrid-INoT**, a hybrid architecture in which the *planner–worker–critic–self-check* role cycle is unfolded inside a single model call, while only tool actions, verification, and context compression remain external. For this architecture, we specify an operational model, the `RouteMode` routing function, and a termination condition for the internal role cycle. We show that, with sufficiently long shared context and several external roles, Hybrid-INoT can be more economical than Classical-MAS in total token usage; in addition, we derive a worst-case upper bound on token consumption and a profitability condition for a small self-check model. The paper also proposes a metric system for token efficiency, monetary cost, and code maintainability, together with an empirical validation program that includes a pilot comparison on HumanEval, an ablation study, a context-length scaling analysis, an evaluation of the self-check procedure, and a TCO analysis based on real measurements.

Keywords: Large language models, agent systems, role introspection, Introspection of Thought, Self-Refine, Reflexion, token efficiency, economics of inference, code reliability, experimental protocol

1 Introduction

A modern software-engineering agent must successively understand the task specification, retrieve the relevant context from a repository, and construct a change that can then be checked by tests or static analysis. In practice, these functions are often distributed across several roles: one role builds a plan, another proposes a solution, a third critiques the intermediate result, and a fourth runs external verification. This decomposition is conceptually convenient, but it becomes expensive when all roles are forced to reread the same long context.

This paper focuses precisely on that setting: tasks in which the code, documentation, compiler messages, and execution logs are shared across all roles. We are interested not only in the final solution quality but also in the cost of the chosen orchestration scheme in tokens, response time, and monetary spending. The article therefore compares classical external multi-agent organization with a hybrid scheme in which part of the role discussion is moved inside a single model call.

1.1 The problem of repeated context transmission and its practical significance

In agent systems built on large language models, a typical configuration is external multi-agent orchestration (Classical-MAS): separate processes or calls implement the planner, worker, critic, and tester roles [12–14]. In software-engineering tasks, where several roles read the same long fragments of code, logs, and documentation, this scheme has a structural defect: *each role receives the shared context* $\mathcal{C}_0$ *anew*, and additional coordination messages circulate between roles. In token-budget terms, this yields the term

$$D_{\text{ctx}} + H_{\text{coord}} \quad \in \quad \Theta(|\mathcal{A}| \cdot |\mathcal{C}_0|) + \Theta(|\mathcal{A}|^2 \cdot \bar{h}),$$

where $|\mathcal{A}|$ is the number of roles and $\bar{h}$ is the average coordination-message length. This is not merely a qualitative observation: empirical measurements show that when $|\mathcal{A}| \geq 4$ and $|\mathcal{C}_0| \geq 4000$ tokens, the context share in the total cost exceeds 60% [14, 15]. At standard API prices of \$3/M input tokens (roughly the level of Claude Sonnet 3.5 / GPT-4o in 2025), the price of this structural defect amounts to thousands of dollars per month even at a modest workload of 10,000 tasks. Here D_{ctx} denotes the cost of repeated shared-context transmission, H_{coord} the cost of coordination messages, $|\mathcal{C}_0|$ the length of the initial context in tokens, $|\mathcal{A}|$ the number of roles, and $\bar{h}$ the average coordination-message length.

Thus, the costs of the architectural scheme under discussion admit a quantitative expression: at the scale of 10^4–10^6 engineering tasks, Classical-MAS spends an order of magnitude more resources on repeated context retransmission than on direct solution generation. It is precisely this fact that determines the relevance of the problem studied here.

1.2 Existing approaches and their limitations

The open literature contains three related lines of work: (i) single-model in-context self-critique, such as Self-Refine [7] and Reflexion [8]; (ii) alternation between reasoning

and action inside one agent, such as ReAct [6] and SWE-agent [17]; and (iii) external role decomposition into separate calls, such as MetaGPT [12] and ChatDev [13]. None of these works defines an *operationally distinct* intermediate configuration in which (a) the shared context is preserved inside one call without duplication, (b) role segregation remains sufficient for critical examination, and (c) only operations with externally verifiable signals, such as tests, static analysis, and tool calls, remain external. The literature also lacks a formal condition under which such a hybrid dominates a purely external Classical-MAS scheme in expected token consumption.

1.3 Main results and contribution

The paper delivers the following results:

C1. An operational definition of Hybrid-INoT with an explicit state signature, the `RouteMode` routing function, and a termination condition for the internal role cycle (§4).

C2. Proposition 2, which states the token advantage of Hybrid-INoT over Classical-MAS in expectation when the shared context is sufficiently long, and proves it by an explicit token accounting under stated assumptions (§5).

C3. Inequality (2), which gives the profitability condition for a small self-check model (§5).

C4. A system of metrics for token efficiency, monetary cost, and code maintainability with a justified aggregation form (§6).

C5. An empirical validation program comprising five studies, E1–E5, with fixed baseline configurations, ablation analysis, and explicitly unfavorable scenarios (§7).

The empirical validation of Hybrid-INoT under long context is discussed further in §10.

2 Related work and distinction of the proposed approach

2.1 From chain-of-thought to internal role debate

Wei et al. [5] show that explicit verbalization of intermediate reasoning steps can improve solution quality on tasks that require several connected logical transitions. In this context, intermediate steps are textual fragments by which the model decomposes a complex inference into a sequence of local subproblems, while quality refers to the final success of solving the target task. The Self-Refine approach [7] formalizes a "generator–feedback–revision" cycle inside a single model; Reflexion [8] supplements such a scheme with reflective memory, whereas ReAct [6] couples internal reasoning with external actions and tools. Sun and Zeng [1] describe *Introspection of Thought* (INoT) as a structured internal debate among several roles. At the same time, Turpin et al. [9] show that intermediate chain-of-thought text should not be interpreted as a faithful description of the hidden computational process. Accordingly, in the present paper introspection is treated primarily as a computational mechanism for candidate

selection, critique, and selective rerun rather than as a literal window into the latent state of the model.

2.2 Multi-agent systems for software engineering

Weiss [10] and Wooldridge [11] provide the theoretical framework for MAS. MetaGPT [12] and ChatDev [13] transfer this framework into the context of large language models: roles are implemented as separate calls, each receiving its own copy of the context. Survey work identifies two key limitations of Classical-MAS: error propagation along the chain and the growth of coordination cost [14, 15]. SWE-bench and SWE-agent [16, 17] further show that practical software-engineering automation requires long context and iterative editing, which makes the retransmission problem D_{ctx} even more acute.

2.3 Context compression and external verification

LongLLMLingua and LLMLingua-2 [20, 21] demonstrate that prompt compression can preserve or even improve quality under long context while lowering cost. Jia et al. [22] apply this logic directly to code. For code self-check, linguistic reflection alone is insufficient: external observable signals are required, namely tests, static analysis, and compiler feedback [23–25].

2.4 Comparison of Hybrid-INoT with prior approaches

To systematize the differences among the architectures under discussion, we distinguish five features: context organization, the structure of role interaction, the verification mechanism, the way the execution mode is chosen, and the stopping condition.

Table 1 shows that individual components of the proposed scheme appear in earlier work, but it is precisely their combination inside a single internal role cycle, supplemented by external tool-based verification and explicit routing, that constitutes the Hybrid-INoT architecture.

3 Formal problem statement

3.1 Task

Let $\mathcal{X}$ denote the set of textual task specifications, $\mathcal{C}$ the set of finite sequences of contextual artifacts (code files, documentation fragments, and execution logs), and $\mathcal{Y}$ the set of candidate solutions representable as textual artifacts and amenable to external verification. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be the probability space induced by stochastic model generation, seed choice, and nondeterminism in external tool calls.

Definition 1 (Agentic engineering task)**.** *An agentic engineering task is the tuple $(x, \mathcal{C}_0, \mathcal{Z}, \mathcal{V})$, where $x \in \mathcal{X}$ is a textual task description; $\mathcal{C}_0 \in \mathcal{C}$ is the initial task context, including code, documentation, and execution logs; $\mathcal{Z}$ is a finite set of permitted*

Table 1 Comparison of Hybrid-INoT with adjacent architectural classes

Feature	Self-Refine / Reflexion	MetaGPT / ChatDev	Hybrid-INoT
Context organization	One shared context without explicit role separation.	A separate copy of the shared context is created for each role.	One shared context is preserved inside a model call and shared among the internal roles.
Role interaction	Self-critique is embedded in a single-model loop.	Roles are separated into distinct calls and exchange messages.	Planning, generation, and critique are executed as internal roles within one cycle.
Solution verification	Predominantly linguistic self-evaluation.	Predominantly role-based linguistic checking; external verification depends on the implementation.	The final candidate must pass external tool-based verification; linguistic critique is used as an intermediate filter.
Mode selection	Fixed by the method architecture.	Fixed by the method architecture.	Set by an explicit routing function that switches between internal and external modes.
Stopping condition	A fixed number of iterations or an episodic stopping rule.	Completion of the role pipeline or agreement among roles.	Stop after successful external verification or after the iteration limit is reached.

external tools; and $\mathcal{V} : \mathcal{Y} \to \{0, 1\}$ *is an externally verifiable success function, for example, the execution of a test suite.*

The agent strategy induces a random output $Y : \Omega \to \mathcal{Y}$. At the same time, random variables $T : \Omega \to \mathbb{R}_{\geq 0}$, $C_{\text{inf}} : \Omega \to \mathbb{R}_{\geq 0}$, and $\ell : \Omega \to \mathbb{R}_{\geq 0}$ are defined, representing, respectively, the total token consumption, the monetary inference cost, and the latency of solving one task. The goal of the agent is to maximize $\mathbb{E}[\mathcal{V}(Y)]$ under the budget constraints $\mathbb{E}[T] \leq B$, $\mathbb{E}[C_{\text{inf}}] \leq B_{\$}$, and $\mathbb{E}[\ell] \leq B_{\ell}$.

Main notation

$\mathcal{C}_0$ — the initial shared task context; $|\mathcal{C}_0|$ is its token length.
$\hat{\mathcal{C}}$ — the compressed version of the initial context used in the internal cycle.
$\mathcal{A}$ — the set of external roles in the Classical-MAS scheme; $|\mathcal{A}|$ is the number of such roles.
Y — the random output of the agent with values in $\mathcal{Y}$; $y \in \mathcal{Y}$ is an individual candidate solution.
K — the number of candidates generated by the role r_{work} in one iteration of the internal cycle.
T — the random variable of total token consumption for one task; $\mathbb{E}[T]$ is taken with respect to the distribution defined on $(\Omega, \mathcal{F}, \mathbb{P})$.
U_{tok} — the token-efficiency measure defined as the ratio of final solution quality to total token consumption.
$\tau^\star$ — the threshold initial-context length beyond which Hybrid-INoT has an expected token advantage.

δ_{H1} — the minimal practically relevant gain in U_{tok} used in Hypothesis H1.
C^S_{check} — the cost of the preliminary check performed by the small model.
C^L_{rerun} — the cost of rerunning the large model.
ρ_0, ρ_1 — the probabilities of an expensive rerun without and with the preliminary check, respectively.

3.2 Research hypotheses

H1 *(advantage under long context).* There exists a threshold initial-context length $\tau^\star$ such that for $|\mathcal{C}_0| > \tau^\star$, the Hybrid-INoT architecture yields at least a $\delta_{H1} = 15\%$ increase in U_{tok} relative to Classical-MAS, while solution quality degrades by no more than 2 percentage points.

Verification: experiment E1 (§7.2) with the paired Wilcoxon signed-rank test, 95% bootstrap confidence intervals, and Holm–Bonferroni correction; the value of $\tau^\star$ is estimated in E3 (§7.4).

H2 *(economic justification of the small self-check model).* There exists a regime in which the preliminary check performed by a small model reduces the probability of an expensive rerun enough for condition (2) to hold: $C^S_{\text{check}} < (\rho_0 - \rho_1)C^L_{\text{rerun}}$.

Verification: E4 (§7.5) with direct measurements of $\rho_0, \rho_1, C^S_{\text{check}}, C^L_{\text{rerun}}$ on 100 HumanEval tasks and 50 SWE-bench Lite tasks.

H3 *(boundary of applicability).* On tasks with genuinely independent subtasks and a high degree of tool parallelism, the response-time advantage of Hybrid-INoT vanishes or becomes negative.

Verification: dedicated experiment E2c (§7.3) on a subset of clearly parallel tasks from the Synthetic Tool-Use Suite.

When testing H1, the base model, the set of allowed tools, the external-verification protocol, the task set, and the iteration limit are held fixed across all compared configurations; only the orchestration scheme and the way the shared context is reused across roles are changed. In H3, genuinely independent subtasks are those for which partial solutions can be constructed without exchanging intermediate state and without jointly reading the shared context, while a high degree of tool parallelism means that external checks and tool calls can be executed concurrently without blocking dependencies. Hypothesis H3 therefore defines the applicability boundary of the architecture: under such conditions, the latency advantage of the internal role cycle may disappear. The threshold $\delta_{H1} = 15\%$ for H1 is chosen as the smallest practically meaningful improvement, taking into account the typical variability of token budgets across seeds ($\approx 5{-}8\%$ std in [7]) and the engineering cost of deploying the hybrid scheme at a scale of 10^4 tasks per month. Sensitivity to this choice is assessed in E1 on the grid $\delta \in \{5\%, 10\%, 15\%, 20\%\}$.

4 Method: Hybrid-INoT

4.1 States and role signatures

Define the system state as $\sigma = (x, \hat{\mathcal{C}}, \mathcal{H}, i, b)$, where $\hat{\mathcal{C}}$ is the compressed context, $\mathcal{H}$ is the history of iterations, $i \in \{0, \dots, I\}$ is the iteration counter, and $b \in \mathbb{N}$ is the remaining token budget.

Let Π be the space of plans, and let $K \in \mathbb{N}$ denote the number of candidates generated by the role r_{work} in one iteration. Then each role is defined by a signature $r : \mathcal{S}_r^{\text{in}} \to \mathcal{S}_r^{\text{out}}$:

$$\begin{aligned} r_{\text{plan}} &: (x, \hat{\mathcal{C}}, b) \mapsto \pi \in \Pi, \\ r_{\text{work}} &: (\pi, \hat{\mathcal{C}}, K) \mapsto \mathbf{y} = (y_1, \dots, y_K) \in \mathcal{Y}^K, \\ r_{\text{crit}} &: (\mathbf{y}, \hat{\mathcal{C}}) \mapsto (s_1, \dots, s_K) \in [0,1]^K, \\ r_{\text{self}} &: (y \in \mathcal{Y}) \mapsto (v, M) \in \{0,1\} \times [0,1]. \end{aligned}$$

The roles $r_{\text{plan}}, r_{\text{work}}, r_{\text{crit}}$ are implemented as role prefixes inside *one* large-model call over the shared $\hat{\mathcal{C}}$; the role r_{self} is an external process running tests and static analysis.

4.2 Routing function

Definition 2 (RouteMode)**.** *Let $\varphi_{\text{tool}}(x) \in \{0,1\}$ indicate whether an external tool is mandatory, let $\varphi_{\text{par}}(x) \in \{0,1\}$ indicate dominant subtask parallelism, and let $|\hat{\mathcal{C}}|$ be the length of the compressed context. Then*

$$\texttt{RouteMode}(x, \hat{\mathcal{C}}) = \begin{cases} \texttt{external}, & \varphi_{\text{tool}}(x) = 1 \ \vee \ \varphi_{\text{par}}(x) = 1 \ \vee \ |\hat{\mathcal{C}}| < \tau^{\star}_{\text{route}}, \\ \texttt{internal}, & \textit{otherwise}. \end{cases}$$

The parameters φ_{tool} and φ_{par} are determined by a separate classifier over x (in the base implementation, a lightweight instruction-based classifier; later, a trained binary classifier); the threshold $\tau^{\star}_{\text{route}} = 1024$ tokens is used as an initial configuration and refined in E3.

4.3 Internal cycle and termination condition

Proposition 1 (Termination of the internal cycle)**.** *Assume that each operation within one iteration of Algorithm 1 terminates in finite time and that the number of iterations is bounded above by the parameter I. Then the algorithm terminates no later than after I iterations: it either returns a solution immediately after a successful external check $v_i = 1$, or stops earlier under the rule* $\text{stagnation}(s_{i,k^\star} - s_{i-1,k^\star}) < \varepsilon$*, or, after exhausting the iteration limit, returns the best candidate found.*

Proof The outer loop over the index i contains at most I passes. Each iteration consists of a finite sequence of steps: plan construction, generation of K candidates, critical scoring, external verification, and context update. By assumption, all of these operations terminate

Algorithm 1 Hybrid-INoT: internal introspective cycle

Require: task x, context $\mathcal{C}_0$, budget B, maximum number of iterations I, number of candidates K

Ensure: solution $y^\star$, flag $v^\star$, score $M^\star$

$\mathcal{C} \leftarrow \texttt{PolicySanitize}(x, \mathcal{C}_0)$
$\hat{\mathcal{C}} \leftarrow \texttt{CompressIfNeeded}(\mathcal{C})$ ▷ Alg. 2
$m \leftarrow \texttt{RouteMode}(x, \hat{\mathcal{C}})$ ▷ Def. 2
if $m = \texttt{external}$ **then return** $\texttt{ExternalAgentWorkflow}(x, \hat{\mathcal{C}})$
end if
$s^{\text{best}} \leftarrow -\infty;\ y^{\text{best}} \leftarrow \perp$
for $i = 1$ to I **do**
  $\pi_i \leftarrow r_{\text{plan}}(x, \hat{\mathcal{C}}, B - \text{spent})$
  $\{y_{i,k}\} \leftarrow r_{\text{work}}(\pi_i, \hat{\mathcal{C}}, K)$
  $(s_{i,1}, \ldots, s_{i,K}) \leftarrow r_{\text{crit}}(\{y_{i,k}\}, \hat{\mathcal{C}})$
  $k^\star \leftarrow \arg\max_k s_{i,k};\ y_i \leftarrow y_{i,k^\star}$
  if $s_{i,k^\star} > s^{\text{best}}$ **then** $s^{\text{best}} \leftarrow s_{i,k^\star};\ y^{\text{best}} \leftarrow y_i$
  end if
  $(v_i, M_i) \leftarrow r_{\text{self}}(y_i)$ ▷ external tests and static analysis
  if $v_i = 1$ **then return** $(y_i, 1, M_i)$
  end if
  $\hat{\mathcal{C}} \leftarrow \texttt{UpdateContextWithFailures}(\hat{\mathcal{C}}, v_i)$
  if $\text{stagnation}(s_{i,k^\star} - s_{i-1,k^\star}) < \varepsilon$ **then break** ▷ see Prop. 1
  end if
end for
return $(y^{\text{best}}, 0, M(y^{\text{best}}))$

in finite time; therefore, each iteration also terminates. Consequently, the algorithm either stops early by the instructions **return** or **break**, or reaches the final **return** after the I-th iteration. Hence the running time of the internal cycle is bounded above by a finite number of iterations, which proves the statement. □

Note

Proposition 1 guarantees only algorithm termination; it does not imply monotonic growth of the scores s_i, nor does it guarantee the correctness of the critic itself. The scenario in which the critic systematically endorses incorrect solutions is examined separately in E2b (§7.3) as one of the threats to validity.

Algorithm 2 Compression of long context

Require: $\mathcal{C}_0$, length threshold τ_{len}
Ensure: $\hat{\mathcal{C}}$

```
if |C_0| ≤ τ_len then return C_0
end if
split C_0 into semantic blocks {b_j}
estimate rel(b_j) using a classifier of the LLMLingua-2 type [21]
remove duplicates, low-information logs, and secondary documentation
retain API signatures, failing traces, editable files, and task constraints
return Ĉ with |Ĉ| ≤ τ_len
```

Algorithm 3 Selective rerun based on verified signals

Require: candidates $\{y_1, \ldots, y_K\}$, scores $\{s_k\}$, rerun budget R
Ensure: $y^\star$

```
sort {y_k} in descending order of s_k; keep the top M ≤ K
for each y_m do
    q_m ← ExternalVerify(y_m)
    if q_m = 1 then return y_m
    end if
end for
if R > 0 then construct the prompt only from verified failure signals; return SelectiveRerun(R − 1)
end if
return arg max_k s_k
```

4.4 Context compression

4.5 Selective rerun

5 Analytical model of token and monetary costs

5.1 Structure of the analytical relations

The analysis in this section includes two types of relations. The first comprises direct balance equalities and inequalities that define the structure of token and monetary costs. The second comprises statements comparing architectures under fixed assumptions about context length, coordination cost, and internal critique. This separation makes it possible to distinguish general accounting relations from results that require additional assumptions.

5.2 Token budgets of Classical-MAS and Hybrid-INoT

Assumption 1. *Among the roles in $\mathcal{A}$ under the Classical-MAS scheme, at least $\alpha|\mathcal{A}|$ roles, with $\alpha \in (0, 1]$, read the shared context $\mathcal{C}_0$ in full.*

Assumption 2. *The length of coordination messages is bounded by $\bar{h}$, the number of iterations in both schemes is I, the compression cost satisfies $T_{cmp} \leq \gamma|\mathcal{C}_0|$ with $\gamma < \alpha|\mathcal{A}|$, and the additional cost of internal critique satisfies $T_{core}^{add} \leq \beta|\hat{\mathcal{C}}|$ with $\beta < \alpha|\mathcal{A}| - \gamma$.*

Proposition 2 (Token advantage under long context)**.** *Under Assumptions 1–2 and $|\mathcal{A}| \geq 3$, there exists $\tau^\star > 0$ such that for $|\mathcal{C}_0| > \tau^\star$,*

$$\mathbb{E}[T_{MAS}] - \mathbb{E}[T_{INoT}] \;\geq\; (\alpha|\mathcal{A}| - \gamma - \beta) \cdot |\mathcal{C}_0| + \Theta(|\mathcal{A}|^2\bar{h}) - T_{check} - \mathbb{E}[T_{rerun}] \;>\; 0.$$

Derivation from the assumptions Let T_{gen} denote the aggregate token contribution of candidate generation and other terms that do not depend linearly on the number of external roles. Then the first inequality follows from a direct decomposition of the Classical-MAS token budget into three parts: repeated transmission of the shared context, coordination exchange among roles, and candidate generation. By Assumption 1, in each of the I iterations, at least $\alpha|\mathcal{A}|$ external roles reread the full context $\mathcal{C}_0$, so

$$T_{\text{ctx}}^{\text{MAS}} \geq I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0|.$$

By Assumption 2, coordination messages contribute an additional term of order $I \cdot |\mathcal{A}|^2\bar{h}$. Therefore,

$$\mathbb{E}[T_{\text{MAS}}] \geq I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0| + I \cdot |\mathcal{A}|^2\bar{h} + T_{\text{gen}}.$$

For Hybrid-INoT, the shared context is compressed once, so $T_{\text{cmp}} \leq \gamma|\mathcal{C}_0|$; the additional cost of internal critique is bounded above by $T_{\text{core}}^{\text{add}} \leq \beta|\hat{\mathcal{C}}| \leq \beta|\mathcal{C}_0|$. The external check and possible reruns contribute the terms T_{check} and $\mathbb{E}[T_{\text{rerun}}]$, while the remaining generation contribution is again absorbed into T_{gen}. Hence

$$\mathbb{E}[T_{\text{INoT}}] \leq I \cdot |\hat{\mathcal{C}}| + T_{\text{cmp}} + T_{\text{core}}^{\text{add}} + T_{\text{check}} + \mathbb{E}[T_{\text{rerun}}] + T_{\text{gen}}, \qquad |\hat{\mathcal{C}}| \leq |\mathcal{C}_0|.$$

Substituting these bounds and grouping the terms that depend linearly on $|\mathcal{C}_0|$ yields a lower bound on the difference between the token budgets. For sufficiently large $|\mathcal{C}_0|$, the linear term with a positive coefficient compensates the constant additions, which establishes the existence of the threshold $\tau^\star$. □

Scope of applicability

Proposition 2 gives a lower bound on the difference between the expected token budgets under Assumptions 1–2. When $|\mathcal{A}| = 2$ (for example, a plan+exec configuration without a separate critic), the sign of this difference may change. Likewise, if $|\mathcal{C}_0| < \tau^\star$, an advantage of Hybrid-INoT is not guaranteed. The value of $\tau^\star$ depends on $\alpha, \beta, \gamma, |\mathcal{A}|$, and I, and must therefore be refined empirically.

5.3 Worst-case upper bound on token usage

Proposition 3 (Worst-case upper bound on Hybrid-INoT token usage)**.** *In the worst case, when all I iterations end with $v_i = 0$,*

$$T_{INoT}^{worst} \;\leq\; T_{cmp} + I \cdot K \cdot (p^L + r^L) + I \cdot T_{check} + I \cdot T_{rerun}^{unit}.$$

The resulting estimate defines an upper bound on additional token costs and is used as a budget restriction under B: as $T \to B$, Algorithm 1 returns the best candidate available.

5.4 Profitability condition for the small self-check model

Inference cost:

$$C_{\text{inf}} = c_L^{\text{in}} \sum_i p_i^L + c_L^{\text{out}} \sum_i r_i^L + c_S^{\text{in}} \sum_i p_i^S + c_S^{\text{out}} \sum_i r_i^S. \tag{1}$$

A small self-check model is economically beneficial if and only if

$$C_{\text{check}}^S < (\rho_0 - \rho_1) \cdot C_{\text{rerun}}^L. \tag{2}$$

The left-hand side of the inequality is the cost of the preliminary check, whereas the right-hand side is the expected savings from reducing the probability of an expensive rerun of the large model. Inequality (2) therefore defines the decision criterion between a cheap preliminary check and a rerun of the large model. Its practical applicability depends on the values $\rho_0, \rho_1, C_{\text{check}}^S$, and C_{rerun}^L, which are to be estimated in E4.

6 Metric system

6.1 Main metrics and the rationale for the aggregation form

$$U_{\text{tok}} = \frac{Q}{T}, \qquad Q_{\$} = \frac{Q}{C_{\text{inf}}}, \qquad \text{VCR} = \frac{1}{N} \sum_i \mathbb{I}[\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1]. \tag{3}$$

For heterogeneous quality, the paper uses a weighted sum over task classes, $Q = \sum_b w_b Q_b$, with weights specified in Table 2.

6.2 Aggregated success metric and its form

The aggregated success metric is defined as

$$\text{MAS}_i = \min(\text{Success}_i, \text{Verified}_i) \cdot M_i^\lambda, \qquad \lambda \in [0, 1], \quad \lambda = 0.5. \tag{4}$$

The rationale for this construction is as follows. First, Success and Verified are necessary conditions for counting a solution as successful: a patch that fails external verification should not receive a high final score. Second, the multiplier M_i^λ models the effect of maintainability on the final assessment; the parameter λ controls the strength of this effect. The baseline configuration uses $\lambda = 0.5$, and the robustness of conclusions to the choice of λ is assessed in E2d.

6.3 Maintainability weights and their rationale

$$M_i = w_{\text{cc}}(1 - \widehat{\text{CC}}_i) + w_{\text{dup}}(1 - \widehat{\text{Dup}}_i) + w_{\text{warn}}(1 - \widehat{\text{Warn}}_i) + w_{\text{doc}}\widehat{\text{Doc}}_i. \tag{5}$$

Table 2 Maintainability weights and their relation to ISO/IEC 25010:2023 subcategories [26]

Parameter	Value	Rationale
w_{cc}	0.35	ISO 25010 Modifiability; cyclomatic complexity [27] is the most direct proxy for maintenance cost.
w_{dup}	0.25	ISO 25010 Reusability; duplication is a key signal of technical debt.
w_{warn}	0.20	ISO 25010 Analysability; warnings are a surface-level yet reliable signal.
w_{doc}	0.20	ISO 25010 Modifiability; documentation coverage is used as a proxy for comprehensibility.

Sensitivity analysis

E2d tests the robustness of the configuration ranking B0–B3 under variations $(w_{\mathrm{cc}}, w_{\mathrm{dup}}, w_{\mathrm{warn}}, w_{\mathrm{doc}}) \pm 20\%$ and $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ in (4). A reversal of the ranking under such variations is treated as evidence that the metric is overly sensitive to the chosen parameterization.

6.4 Reliability metrics

$$\mathrm{SCR} = \frac{\#\{\text{incorrect candidates detected by the self-check procedure}\}}{\#\{\text{all incorrect candidates}\}}, \tag{6}$$

$$\mathrm{RFR} = \frac{\#\{\text{patches without rollback}\}}{\#\{\text{applied patches}\}}. \tag{7}$$

7 Empirical validation program

7.1 General rules

All experiments fix task identifiers, seed values, model versions with snapshot identifiers, and token prices together with the date of measurement. For closed APIs without a fixed version identifier, the data are moved to a separate appendix describing the observed characteristics. Statistical analysis includes the paired Wilcoxon signed-rank test for continuous metrics, McNemar's test for binary outcomes, 95% bootstrap confidence intervals (10^4 resamples), and Holm–Bonferroni correction.

7.2 E1 — Pilot on HumanEval

Goal: test H1 on a limited sample. **Data:** HumanEval (164 tasks) [18]; for each task, four versions with artificial historical context of length $\{512, 2048, 8192\}$ tokens generated by a deterministic template are considered, for a total of $164 \times 4 = 656$ cases. Seed values: $\{42, 123, 456, 789, 1337\}$. **Compared configurations:**

- B0 Single-Large (one large-model call without iterations);
- B1 Single-Large + Self-Refine [7], three iterations;
- B2 Classical-MAS (planner, worker, critic as separate calls, in a MetaGPT-like architecture);
- B3 Hybrid-INoT ($K = 3$, $I = 3$).

Metrics: pass@1 (share of tasks solved on the first attempt), T (total token consumption), U_{tok}, C_{inf} (USD per 1000 tasks at prices dated {date}), $Q_{\$}$, and response time. **Success criterion for H1:** B3 yields $\Delta U_{\text{tok}} \geq 15\%$ relative to B2 on the subset with $|\mathcal{C}_0| > 2048$, with $p < 0.05$ after Holm–Bonferroni; pass@1 does not degrade by more than 2 percentage points. **Failure criterion:** if these conditions are not met, H1 is treated as unsupported, and the conclusions are stated without claiming an advantage of Hybrid-INoT.

Table 3 defines the reporting format for the results once the experiment is carried out.

Table 3 Reporting template for experiment E1

Configuration	pass@1	tokens/task	U_{tok}	USD/1000 tasks
B0	—	—	—	—
B1 Self-Refine	—	—	—	—
B2 Classical-MAS	—	—	—	—
B3 Hybrid-INoT	—	—	—	—

7.3 E2 — Ablation analysis and unfavorable scenarios

E2a. B3 vs. B3a (without compression), B3b (without critic), and B3c (without self-check). Data: HumanEval, $n = 164$. Criterion: each component must make a statistically significant contribution; if $\Delta_{\text{B3}-\text{B3b}}$ is insignificant, the critic role is removed from the architecture. **E2b** (unfavorable scenario: systematic critic error). A subset of 50 tasks on which the baseline B0 fails; the study measures how often r_{crit} approves an incorrect candidate; an alarm threshold of $> 20\%$ implies the need for external critic training. **E2c** (unfavorable scenario: parallel tools, H3). Fifty tasks with explicitly independent subtasks from the Synthetic Tool-Use Suite; response time is compared between B3 and B2; a response-time degradation of B3 up to 30% is considered acceptable. **E2d** (metric sensitivity analysis). Ranking of configurations under $w \pm 20\%$ and $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$.

7.4 E3 — Scaling with context length

Goal: estimate empirically the threshold $\tau^\star$ from Prop. 2. **Data:** 50 HumanEval tasks $\times$ 4 context lengths $\{256, 1024, 4096, 16384\} = 200$ cases. **Method:** construct $U_{\text{tok}}(|\mathcal{C}_0|)$ for B0, B2, and B3; identify the intersection point $U^{B3}_{\text{tok}}(|\mathcal{C}_0|) = U^{B2}_{\text{tok}}(|\mathcal{C}_0|)$. **Criterion:** $\tau^\star$ must be finite and < 16384; otherwise H1 and Prop. 2 are not confirmed in a practically relevant range.

7.5 E4 — Evaluation of the self-check procedure and H2

Goal: test condition (2). **Data:** 100 HumanEval tasks + 50 SWE-bench Lite tasks [16]. **Measurements:** ρ_0 (share of incorrect solutions without self-check), ρ_1 (with self-check), C^S_{check}, and C^L_{rerun}, obtained by direct measurement. **Additionally:**

SCR recall and the cost-benefit ratio of the self-check procedure. **Criterion for H2:** $C^S_{\text{check}} < (\rho_0 - \rho_1)C^L_{\text{rerun}}$ on both datasets with 95% confidence intervals.

7.6 E5 — TCO on real data

Goal: replace the synthetic TCO calculation with observed measurements. **Data:** $N = 500$ tasks with varying context length, under B2 and B3. **Measurements:** input and output tokens for each role, number of API calls, number of reruns. C_{inf} is computed using current prices, for which the model version and measurement date are recorded. **Extrapolation:** to 10,000 tasks/month with confidence intervals, *not* a point estimate. **Sensitivity analysis:** token-price variation by ±50%. **Criterion:** if even the lower CI bound of ΔTCO is not negative, no economic gain may be claimed.

8 Economic model

8.1 TCO formulation

$$\text{TCO}_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}. \tag{8}$$

Equation (8) defines the structure of total cost of ownership; its parameterization is based on the measurements described in E5.

8.2 Conditions of economic viability

- **Economically favorable:** $|\mathcal{C}_0| > \tau^\star$; $|\mathcal{A}| \geq 3$; the share of errors caught by the cheap self-check procedure is at least 30%; no dominant parallelism is present.
- **Economically unfavorable:** $|\mathcal{C}_0| < \tau^\star$; genuine subtask independence; a requirement of process-level privilege isolation; or poor adherence of the model to the role format (measured in E2b).

These regimes are quantified in experiments E1, E3, and E5.

9 Limitations and threats to validity

9.1 When internal introspection is worse than external orchestration

1. **Genuine subtask independence.** Parallel execution of several large-model calls yields linear speedup in response time, whereas the internal cycle is sequential.
2. **Need for privilege isolation.** Execution of untrusted code and secret handling require process boundaries between roles, which rules out the internal mode.
3. **Weak adherence to the role format.** If the model does not preserve role prefixes, the critic effectively degenerates into a generator with a different instruction, and the architectural advantage disappears. This risk is assessed in E2b.
4. **Systematic critic error.** A critic relying on its own generations may consistently approve incorrect candidates. In that case, the main safeguard remains external verification.

5. **Compression that removes a critical fact.** Algorithm 2 may omit a rare but essential fragment; therefore, failing traces and API signatures are always preserved in the compressed context.

9.2 Threats to validity

Internal validity: the results may be affected by seed choice and API drift; to reduce this effect, model-version identifiers are fixed and a five-seed setup is used. **External validity:** HumanEval contains short synthetic tasks, so transfer of conclusions to real software-engineering tasks is further checked on SWE-bench Lite in E4. **Construct validity:** the metrics U_{tok} and $Q_{\$}$ are sensitive to token prices; this factor is analyzed in E5 under price variations of $\pm 50\%$. **Statistical validity:** multiple comparisons are handled by Holm–Bonferroni correction, and when n is small, the main emphasis is placed on confidence intervals.

9.3 Scope of applicability

The theoretical conclusions obtained here apply to software-engineering tasks with long shared context and are not automatically transferable to other task classes. Intermediate role texts are treated as a computational instrument rather than an interpretable explanation of model behavior [9]. Finally, the specific hyperparameters $K = 3$, $I = 3$, and $\lambda = 0.5$ are used as a working configuration and may be optimized further in later studies.

10 Directions for further research

Further development of this work is primarily connected with full empirical validation of the proposed architecture and refinement of the conditions under which it is applicable.

10.1 Immediate tasks

R1. Implement a reproducible prototype of configurations B0 and B3 with end-to-end execution on HumanEval.

R2. Conduct a pilot comparison on HumanEval according to protocol E1 and obtain the first estimates of pass@1, token consumption, and cost.

R3. Perform the ablation analysis E2a–E2d, including the contribution of compression, the critic, the self-check procedure, and the sensitivity of the metrics.

R4. Refine the distinction from Self-Refine either by strengthening the theoretical analysis or by a direct empirical comparison in terms of U_{tok} under long context.

R5. Estimate TCO from real measurements for a sample of $N = 500$ tasks with confidence intervals and sensitivity analysis with respect to token prices.

R6. Calibrate the weights in Table 2 on the basis of expert elicitation or additional normative sources while preserving the sensitivity analysis.

10.2 Extension of the experimental program

R7. Carry out E3 and E4 in full and obtain an empirical estimate of the threshold $\tau^\star$ with confidence intervals.

R8. Extend the set of baseline configurations with Reflexion [8], ReAct [6], and SWE-agent [17].

R9. Extend the evaluation from SWE-bench Lite to SWE-bench Full on a sample of at least 300 tasks.

R10. Conduct an expert code-quality assessment on a subset of 50 tasks in order to validate the metric M_i against professional judgment.

R11. Study error-propagation mechanisms and compare the effect of a faulty critic in Hybrid-INoT and Classical-MAS.

10.3 Target outcomes of the empirical validation

The central claim about the advantage of internal role segregation under long shared context may be treated as confirmed only if all of the following conditions hold simultaneously:

1. E1: $\Delta U_{\text{tok}}^{B3-B2} \geq 15\%$ on $|\mathcal{C}_0| > 2048$, with $p < 0.05$ after Holm–Bonferroni correction.
2. E1: pass@1 for B3 is not worse than B2 within a non-inferiority margin of 2 percentage points.
3. E2a: each component (compression, critic, self-check) makes a significant independent contribution.
4. E3: $\tau^\star$ is finite and $< 16{,}384$.
5. E4: $C_{\text{check}}^{S} < (\rho_0 - \rho_1) C_{\text{rerun}}^{L}$ on both datasets with 95% confidence intervals.
6. E5: $\Delta\text{TCO} < 0$ at the lower CI bound and robust under $\pm 50\%$ token prices.
7. R4: either a proof or empirical significance against Self-Refine is obtained.

If only part of these conditions is met, the scope of the conclusions must be restricted to the results actually obtained.

10.4 Research focus

Future research should focus on formal and empirical distinction between Hybrid-INoT and external multi-agent orchestration in terms of token efficiency on software-engineering tasks with long context. The maintainability metric system and TCO analysis serve here as supporting components of the broader research program.

11 Conclusion

The paper formalizes Hybrid-INoT as an intermediate architecture between in-context self-critique and external multi-agent orchestration. For this architecture, the paper defines the system state, the routing function, and the internal role cycle; derives a proposition on token advantage under long context and at least three roles; derives a worst-case upper bound on token usage; and proposes a profitability condition for the small self-check model. In addition, it develops a metric system for token

efficiency, monetary cost, and code maintainability and formulates an empirical validation program required for quantitative testing of the theoretical conclusions. Taken together, these results provide a formal basis for further comparison of Hybrid-INoT with Classical-MAS and related approaches on software-engineering tasks.

References

[1] H. Sun and S. Zeng, "Introspection of Thought Helps AI Agents," arXiv preprint arXiv:2507.08664, 2025.

[2] A. J. Ko, R. DeLine, and G. Venolia, "Information Needs in Collocated Software Development Teams," in *Proc. 29th Int. Conf. Software Engineering*, 2007, pp. 344–353.

[3] C. Parnin and S. Rugaber, "Resumption Strategies for Interrupted Programming Tasks," *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.

[4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, "Collaboration, Information Seeking and Communication," *J. Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.

[5] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *NeurIPS*, vol. 35, 2022, pp. 24824–24837.

[6] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," *ICLR*, 2023.

[7] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," *NeurIPS*, 2023.

[8] N. Shinn *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," *NeurIPS*, 2023.

[9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," *NeurIPS*, 2023.

[10] G. Weiss, *Multiagent Systems*. MIT Press, 1999.

[11] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.

[12] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *ICLR*, 2024.

[13] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," *ACL*, 2024.

[14] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," arXiv:2402.01680, 2024.

[15] C. Qian *et al.*, "Scaling Large Language Model-based Multi-Agent Collaboration," arXiv:2406.07155, 2024.

[16] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," *ICLR*, 2024.

[17] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," *NeurIPS*, 2024.

[18] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[19] J. Austin *et al.*, "Program Synthesis with Large Language Models," arXiv:2108.07732, 2021.

[20] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," *ACL*, 2024.

[21] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," *ACL Findings*, 2024.

[22] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing Code Context for LLM-based Issue Resolution," arXiv:2603.28119, 2026.

[23] Z. Bi *et al.*, "Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback," arXiv:2403.16792, 2024.

[24] C.-Y. Su and C. McMillan, "Do Code LLMs Do Static Analysis?," arXiv:2505.12118, 2025.

[25] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, "Enhancing LLM-Based Code Generation with Complexity Metrics," arXiv:2505.23953, 2025.

[26] ISO/IEC 25010:2023, "Systems and software engineering — SQuaRE — Product quality model," ISO, 2023.

[27] T. J. McCabe, "A Complexity Measure," *IEEE Trans. Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.